



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección: 1

Tema 1 - Evolución de los Diagramas T

Profesor:

Felix Marquez

Alumnos:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852

Puerto Ordaz, 08 de Mayo de 2026.

Introducción

El desarrollo de las herramientas de traducción y compilación ha sido un pilar fundamental en la evolución de la informática, permitiendo que la lógica binaria de las máquinas converja con la complejidad del lenguaje humano. Este informe analiza la evolución de los Diagramas en T, una herramienta gráfica esencial para el diseño de traductores, situándolos en el contexto del Procesamiento del Lenguaje Natural (PLN).

A través de este recorrido, se exploran tres grandes etapas:

- La Era Simbólica: Caracterizada por sistemas basados en reglas lógicas y gramáticas formales, donde los Diagramas T representaban procesos deterministas y rígidos.
- La Revolución Estadística (1990-2010): Periodo en el que la "revolución probabilística" transformó los componentes del Diagrama T, pasando de una ingeniería de reglas manuales a modelos basados en datos y probabilidades.
- La Era de la Atención y los Transformers (2018-Presente): Una etapa de cambio de paradigma total impulsada por el Deep Learning y los mecanismos de auto-atención, donde los Diagramas T ahora ayudan a visualizar sistemas capaces de generar y optimizar código de manera inteligente y contextual.

El desarrollo de las herramientas de traducción y compilación ha atravesado diversas etapas fundamentales. Para comprender el papel de los Diagramas en T dentro de la era simbólica, es imperativo establecer primero una base sólida sobre el campo que permitió estas innovaciones: el Procesamiento del Lenguaje Natural.

El Procesamiento del Lenguaje Natural es una rama interdisciplinaria de la inteligencia artificial y la lingüística computacional que se ocupa de la interacción entre las computadoras y los lenguajes humanos. Su objetivo principal es permitir que las máquinas comprendan, interpreten, generen y deriven significado del lenguaje humano de una manera valiosa y eficiente.

Importancia del PLN

La importancia del PLN radica en su capacidad para cerrar la brecha de comunicación entre la lógica binaria de las computadoras y la complejidad ambigua de la comunicación humana. Sin estas técnicas, sería imposible gestionar la enorme cantidad de datos no estructurados en forma de texto que se generan diariamente, ni interactuar con sistemas tecnológicos de forma intuitiva.

Técnicas Principales

Para que una máquina "entienda" el lenguaje, se aplican diversas técnicas organizadas en niveles:

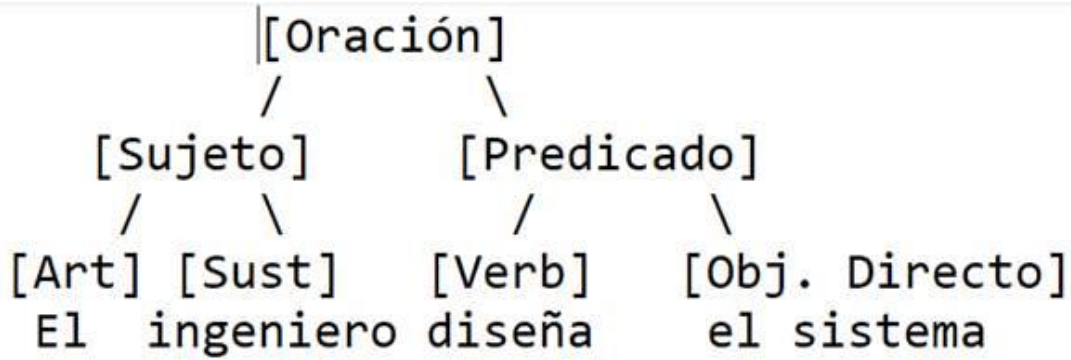
- **Análisis Morfológico y Léxico:** Consiste en la fragmentación de oraciones en unidades mínimas (componentes léxicos) y la identificación de su estructura interna.
- **Etiquetado Gramatical:** Proceso de asignar una categoría (sustantivo, verbo, adjetivo) a cada palabra según su contexto.

- **Lematización y Derivación:** Reducción de las palabras a su raíz o forma básica para normalizar el vocabulario.
- **Análisis Sintáctico:** Determinación de la estructura jerárquica de una oración mediante la aplicación de reglas gramaticales.
- **Análisis Semántico:** Extracción del significado real de las frases, gestionando la ambigüedad y el contexto.

Técnicas y Ejemplos Prácticos

Para procesar una oración, el sistema descompone el texto en niveles. Veamos el ejemplo con la frase: *"El ingeniero diseña el sistema"*.

Técnica	Descripción	Ejemplo Visual / Resultado
Tokenización	Divide el texto en unidades (palabras).	["El", "ingeniero", "diseña", "el", "sistema"]
Etiquetado (POS)	Identifica categorías gramaticales.	[El (Art), ingeniero (Sust), diseña (Verb)...]
Lematización	Lleva la palabra a su raíz base.	"diseña" $\rightarrow$ diseñar
Análisis Sintáctico	Crea un árbol de dependencias.	(Ver Imagen 1 más abajo)



Aplicaciones Actuales

El PLN es el motor detrás de tecnologías cotidianas como:

1. **Sistemas de Traducción Automática:** Conversión de texto entre diferentes idiomas.
2. **Análisis de Sentimientos:** Evaluación de la polaridad emocional en textos (opiniones, redes sociales).
3. **Asistentes Virtuales:** Comprensión de comandos de voz para ejecutar tareas.
4. **Resumen Automático:** Generación de versiones condensadas de documentos extensos.

Ejemplos:

- **Filtros de Spam:** Identifican palabras clave en correos para bloquear publicidad.
- **Correctores Ortográficos:** Sugieren cambios basados en el contexto de la oración.
- **Traducción Automática:** (Ej. Google Translate) que convierte estructuras de un idioma a otro.

La Era Simbólica: Sistemas Basados en Reglas y Gramáticas

En la historia de la computación, la era Simbólica representa el primer gran paradigma del procesamiento del lenguaje y la construcción de traductores. Este enfoque se basa en la premisa de que la inteligencia y el lenguaje pueden representarse mediante la manipulación de símbolos siguiendo reglas lógicas explícitas.

El Corazón de la Era Simbólica: Las Gramáticas Formales

Durante este periodo, el trabajo de lingüistas como Noam Chomsky fue fundamental. Se introdujeron las Gramáticas Libres de Contexto, que permitieron definir la estructura de los lenguajes de programación y los lenguajes naturales de manera matemática.

En esta etapa, un sistema no "aprendía" de los datos (como ocurre hoy con el aprendizaje profundo), sino que dependía de un conjunto exhaustivo de reglas escritas por expertos. Estas reglas dictaban cómo una estructura simbólica (el lenguaje fuente) debía transformarse en otra (el lenguaje objeto).

Sistemas Basados en Reglas

Los sistemas de esta era operaban bajo una lógica de "Si - Entonces". Por ejemplo, si el sistema detectaba una estructura "Sustantivo + Verbo", la regla de producción determinaba cómo debía procesarse esa secuencia para su traducción o ejecución.

- **Ventajas:** Eran sistemas altamente predecibles, transparentes y fáciles de depurar, ya que cada decisión del sistema correspondía a una regla lógica establecida.
- **Limitaciones:** Carecían de flexibilidad. Ante una entrada que no cumpliera estrictamente con las reglas predefinidas, el sistema fallaba. Además, el lenguaje humano es demasiado complejo para ser capturado totalmente por reglas manuales.

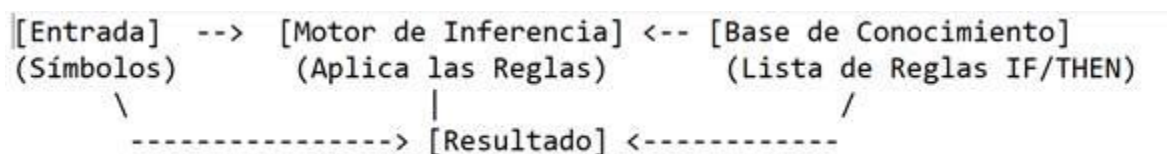
Antes de que existiera el aprendizaje profundo, la informática vivió la era Simbólica. En este periodo, se creía que si podíamos escribir todas las reglas de un idioma en un manual gigante, la computadora podría "entenderlo".

Sistemas Basados en Reglas

Estos sistemas funcionan con lógica pura. No aprenden por experiencia, sino por instrucciones explícitas.

Ejemplo de Regla de Traducción (Inglés a Español):

- **Regla:** *IF* (Sustantivo + Adjetivo en Inglés) → *THEN* (Sustantivo + Adjetivo en Español, invirtiendo el orden).
- **Entrada:** "Red car"
- **Proceso:** El sistema detecta "Red" (Adj) y "Car" (Sust). Aplica la regla de inversión.
- **Salida:** "Carro rojo"



Limitaciones de la Era Simbólica

Aunque estos sistemas eran muy precisos para tareas matemáticas, fallaban con el lenguaje humano debido a la ambigüedad. Por ejemplo, la palabra "banco" puede ser un asiento o una institución financiera; un sistema de reglas rígido tendría dificultades para decidir cuál usar sin un contexto masivo.

Para profundizar en los sistemas basados en reglas, es vital mencionar el uso extensivo de diccionarios computacionales estructurados. Estos vocabularios almacenaban no solo las palabras, sino sus atributos morfológicos detallados. Así, el motor de inferencia consultaba estas estructuras para validar cada elemento de la oración. Esto requería un esfuerzo humano colosal para catalogar cada variación lingüística posible. El sistema dependía totalmente de la exhaustividad de este catálogo manual.

Otra pieza clave de esta era fue el desarrollo de analizadores sintácticos que construían árboles de derivación lógicos. Estos algoritmos evaluaban la oración buscando coincidencias exactas con las gramáticas formales definidas previamente. Si la oración contenía una estructura no prevista en las reglas, el analizador simplemente se detenía y arrojaba un error. No existía la capacidad de inferir el significado por aproximación o similitud. La rigidez era la norma absoluta en la validación del texto.

La construcción de estos sistemas se realizaba mediante lenguajes de programación declarativos centrados en la resolución lógica. Estos lenguajes permitían a los ingenieros definir hechos y reglas sobre el lenguaje natural de forma directa. El motor de búsqueda interno intentaba encontrar un camino lógico que conectara la entrada con una traducción válida. Sin embargo, a medida que el vocabulario y las reglas crecían, el tiempo de procesamiento aumentaba considerablemente. Esto provocaba graves retrasos en la ejecución de traducciones largas.

A pesar de estas severas limitaciones de rendimiento y adaptabilidad, la era simbólica dejó un legado estructural invaluable. Obligó a los investigadores a categorizar y entender el lenguaje humano desde una perspectiva estrictamente formal. Los cimientos teóricos contruidos durante estas décadas son los que permitieron organizar las bases de datos modernas. Sin este primer intento de formalización estricta, los sistemas actuales carecerían de herramientas para validar la coherencia gramatical.

El Rol de los Diagramas en T en la Era Simbólica

Los Diagramas en T surgieron como una herramienta gráfica esencial para visualizar y diseñar la evolución de estos sistemas simbólicos. En un entorno donde los lenguajes de programación estaban naciendo, estos diagramas permitían representar la relación entre tres lenguajes distintos en un proceso de traducción.

Representación Estructural

Un Diagrama en T típico en la era simbólica se divide en tres secciones:

1. **Lado Izquierdo (Lenguaje Fuente):** El lenguaje original en el que está escrito el programa o texto.
2. **Lado Derecho (Lenguaje Objeto):** El lenguaje al cual se desea traducir.
3. **Base de la T (Lenguaje de Implementación):** El lenguaje en el que está escrito el propio traductor o compilador.

El Auge de la Probabilidad: Modelos estadísticos (1990 - 2010)

Durante el periodo 1990 - 2010, las ciencias de la computación vivieron la llamada "revolución estadística". Esto transformó por completo la utilidad de los diagramas T al pasar de una ingeniería de software basada en reglas puras y gramáticas formales estrictas (años 60-80) a modelos probabilísticos y basados en datos, especialmente en el campo de la traducción automática estadística (SMT).

1. Redefinición Probabilística de los Diagramas T

En los años 60, el diagrama T tradicional asumía una traducción determinista (si el código fuente tenía un error sintáctico, fallaba). En la era 1990-2010, los componentes de la "T" pasaron a ser probabilísticos:

- **Lenguaje Fuente (S):** Ya no es solo un código fuente de programación; pasa a ser una cadena de texto o texto libre en un idioma natural.
- **Lenguaje Objetivo (O):** Se usa el Modelo de Canal Ruidoso para obtener la representación más cercana al lenguaje objetivo usado probabilidad condicionada.

$$\hat{O} = \arg \max_O P(O|S) = \arg \max_O P(S|O) \cdot P(O)$$

- **Implementación (I):** La herramienta de traducción ya no es un compilador estático escrito en C o Pascal. Es un Decodificador Estadístico impulsado por corpus lingüísticos paralelos alineados.

2. Modelos Estadísticos Clave (1990 - 2010)

Durante estas dos décadas, los motores estadísticos que daban vida a la base de estas transiciones evolucionaron en tres fases:

- **Modelos Basados en Palabras (Años 1990)**

- **Modelos IBM (del 1 al 5):** Se centraban en la traducción de palabras individuales. Usaban variables ocultas para cuadrar la alineación de idiomas.
 - **Modelo 1 (Léxico Puro):** Solo considera la probabilidad de que una palabra en un idioma de entrada se traduzca como otra en el idioma destino. Traducción palabra por palabra.
 - **Modelo 2 (Posición):** Presta atención a la posición de las palabras en un idioma y las considera para el idioma destino.
 - **Modelo 3 (Fertilidad):** Contempla la probabilidad de que una palabra se convierta en 2 palabras en el idioma destino
 - **Modelo 4 (Alineación relativa):** Posiciones relativas. Los grupos de palabras suelen moverse juntos. En bloques.
 - **Modelo 5 (Deficiencia):** Mejora de todos los anteriores, evitando desperdiciar porcentajes de probabilidad en estructuras de frases imposibles.

Palabra fuente (S) -----> Palabra destino (O)

v

Motor de traducción

- **Modelos Basados en Frases (Años 2000)**

- **Auge de Moses (2007):** De palabra a palabra se pasó a frases. Sistema de código abierto que permitió que el motor de traducción maneje bloques de texto. Se convirtió en el estándar. Captura mejor el contexto inmediato. El sistema

Moses calcula la probabilidad de que un bloque de texto se traduzca como otro bloque en el idioma destino. Se busca que el resultado sea fluido y coherente.

Frase fuente (S) -----> Frase destino (O)

v

Motor de traducción: Moses

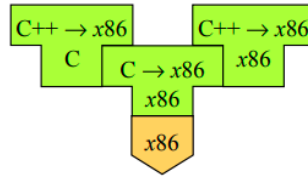
- **Modelos Basados en Sintaxis y Árboles (Finales de los 2000)**
 - **Gramáticas Libres de Contexto Probabilísticas (PCFG):** Los diagramas T comenzaron a representar sistemas que mapeaban estructuras de árboles sintácticos entre (S) y (O). Se visualizaban jerarquías gramaticales (Sujeto, verbo, etc) de ambos idiomas. Muestran el reposicionamiento de palabras cuando los idiomas tienen estructuras muy diferentes como pasar de ingles a japones.
 - **Árbol a Cadena:** Se analiza la sintaxis del idioma fuente para formar un árbol, el idioma destino solo se crea una cadena de texto.
 - **Cadena a árbol:** El idioma entrada es texto plano pero el decodificador genera de forma simultánea la traducción y el árbol sintáctico del destino.
 - **Árbol a árbol:** Entrada y salida se representan mediante árboles. Ambos son complejos.

Árbol fuente (S) -----> Árbol destino (O)

v

Motor de traducción

Compilando un Compilador

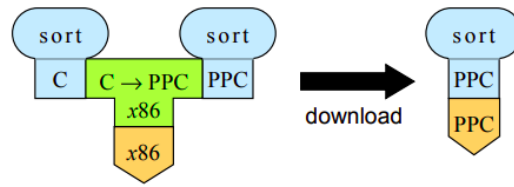


3. El Impacto de la Época en la Práctica

Este enfoque probabilístico eliminó la necesidad de codificar a mano miles de reglas gramaticales abstractas. Si se disponía de suficientes datos empíricos (como las actas del Parlamento Europeo o de las Naciones Unidas), el modelo estadístico de la "T" aprendía automáticamente a traducir combinando un Modelo de Traducción $P(S/O)$ con un Modelo de Lenguaje $P(O)$. Esta arquitectura dominó la computación hasta la llegada de las redes neuronales profundas en la década de 2010. Además, los diagramas T permitieron consolidar técnicas fundamentales:

- **Formalización del Bootstrapping:** Se perfeccionó el uso de un subconjunto de un lenguaje para escribir su propio compilador.
- **Aparición de Lenguajes Intermedios:** Se popularizó dividir el proceso en **Front-end** (Source $\rightarrow$ Intermediate) y **Back-end** (Intermediate $\rightarrow$ Target), utilizando representaciones como el P-Code de Pascal.
- **Traducción Cruzada (Cross-Compilation):** Los diagramas T permitieron definir cómo una máquina con arquitectura X genera código para una arquitectura Y, algo vital para sistemas embebidos.

Compilación Cruzada



¿Qué es embedding?

Embedding es un medio para representar objetos como texto, imágenes y audio como puntos en un espacio vectorial continuo donde las ubicaciones de esos puntos en el espacio son semánticamente significativas para los algoritmos de machine learning (ML). Es una herramienta crítica para los ingenieros de ML que construyen motores de búsqueda de texto e imágenes, sistemas de recomendación, chatbots, sistemas de detección de fraude y muchas otras aplicaciones. En esencia, embedding permite a los modelos de machine learning encontrar objetos similares.

A diferencia de otras técnicas de ML, los embeddings se aprenden a partir de datos utilizando varios algoritmos, como redes neuronales, en lugar de requerir explícitamente experiencia humana para definirlos. Permiten que el modelo aprenda patrones y relaciones complejas en los datos, que de otro modo sería imposible para los humanos identificar.

¿Cómo funciona el embedding?

La mayoría de los algoritmos de machine learning solo pueden recibir datos numéricos de baja dimensión como entradas. Por lo tanto, es necesario convertir los datos a un formato numérico. Esto puede implicar cosas como crear una representación de “bag of words” para datos de texto, convertir imágenes en valores de píxeles o transformar datos de grafos en una matriz numérica. Los objetos que ingresan a un modelo de embedding se generan como embeddings, representados como vectores. Un vector es un arreglo de números (por ejemplo, 1489, 22... 3, 777), donde cada número indica dónde se encuentra un objeto a lo largo de una dimensión específica. El número de dimensiones puede llegar a mil o más dependiendo de la complejidad de los datos de entrada. Cuanto más cerca esté un embedding de otros

embeddings en este espacio n-dimensional, más similares serán. La similitud de distribución se determina por la longitud de los puntos vectoriales de un objeto a otro.

Word2Vec

Desarrollado por Google en 2013, es un método para crear eficientemente embeddings de palabras utilizando una red neuronal de dos capas. Toma como entrada una palabra y produce una coordenada n-dimensional (el vector embedding) de modo que, cuando se grafican estos vectores de palabras en un espacio tridimensional, los sinónimos se agrupan.

Aquí se muestra cómo dos palabras, “papá” y “mamá”, serían representadas como vectores:

“papá”=[0.1548,0.4848,...,1.864]

“mamá”=[0.8785,0.8974,...,2.794]

Aunque existe cierta similitud entre estas dos palabras, esperaríamos que “padre” estuviera mucho más cerca de “papá” en el espacio vectorial, lo que resultaría en un producto punto más alto.

¿Cómo se crean los embeddings?

Los embeddings se crean mediante un proceso llamado "embedding learning". Aunque el método específico utilizado depende del tipo de datos que se estén representando, los embeddings se crean siguiendo estos pasos generales:

- **Elegir o entrenar un modelo de embedding:** Seleccionar un modelo de embedding preexistente adecuado para tus datos y tarea, o entrenar uno nuevo si es necesario. Para texto, se puede elegir Word2Vec, GloVe o BERT.
- **Preparar tus datos:** Dar formato a tus datos de una manera compatible con el modelo de embedding elegido. Para texto, esto implica tokenización y posiblemente preprocesamiento. Para imágenes, puede ser necesario redimensionar y normalizar las imágenes.
- **Cargar o entrenar el modelo de embedding:** Si se utiliza un modelo preentrenado, cargar los pesos y la arquitectura. Si se entrena un nuevo modelo, proporcionar los datos de entrenamiento preparados al algoritmo.
- **Generar embeddings:** Para cada punto de datos, utilizar el modelo entrenado o cargado para generar embeddings. Por ejemplo, si se utiliza un modelo de embedding de palabras, introducir una palabra para obtener su vector correspondiente.
- **Integrar embeddings en tu aplicación:** Utilizar los embeddings generados como características en tu modelo de machine learning, o para búsqueda por similitud, recomendación, clustering, etc., dependiendo de tu tarea específica.

En todos los casos de embedding, la idea es representar datos en un espacio vectorial continuo donde se preserven relaciones significativas.

Procesamiento de Lenguaje Natural (NLP)

Embeddings de palabras en análisis de sentimientos: Los embeddings de palabras como Word2Vec o GloVe se utilizan para representar palabras en un espacio vectorial continuo.

Los modelos de análisis de sentimientos pueden aprovechar estos embeddings para comprender y clasificar el sentimiento de un texto.

BERT para preguntas y respuestas: Los embeddings de BERT se utilizan en sistemas de preguntas y respuestas. El modelo puede comprender el contexto de la pregunta y del documento para extraer información relevante.

Similitud de texto con Doc2Vec: Los embeddings de Doc2Vec se aplican en tareas como encontrar documentos similares. Los embeddings de documentos se comparan para medir la similitud semántica entre documentos.

La evolución del Procesamiento de Lenguaje Natural (PLN) representa uno de los cambios de paradigma más profundos en la informática: la transición de un enfoque simbólico y prescriptivo hacia uno probabilístico y neuronal.

En sus inicios, el PLN compartía la misma base que la teoría de compiladores tradicional, donde el lenguaje se entendía mediante reglas sintácticas estrictas y autómatas finitos. Sin embargo, la naturaleza ambigua y cambiante del lenguaje humano demostró que los sistemas basados puramente en reglas eran insuficientes. El nacimiento de los embeddings y la revolución del Deep Learning permitieron que las máquinas dejaran de ver el lenguaje como una secuencia de tokens aislados, para empezar a procesarlo como un espacio matemático continuo donde el contexto y la semántica son lo más importante.

La Era de la Atención y los Transformers

En el procesamiento de lenguaje natural (NLP) y la teoría de lenguajes y compiladores, el periodo que comenzó en 2018 representa un cambio de paradigma total. Esta etapa, conocida como la Era de la Atención, ha desplazado las arquitecturas secuenciales (como RNN y LSTM) en favor de los Transformers.

El Hito Fundacional: "Attention is All You Need" (2017/2018)

Aunque el artículo de Google se publicó a finales de 2017, su impacto comenzó en 2018.

El Mecanismo de Autoatención (Self-Attention): A diferencia de los compiladores tradicionales que analizan el código de forma lineal (token por token), el Transformer puede observar toda la secuencia simultáneamente.

- **Paralelismo:** Eliminó la dependencia de "pasos temporales" de las redes recurrentes, permitiendo entrenar modelos masivos en GPUs.
- **Contexto Global:** Permite capturar relaciones entre palabras o variables que están muy alejadas en el texto o código, resolviendo el problema de las dependencias a largo plazo.

Evolución de Modelos (2018 - Presente)

Esta era se divide en tres grandes corrientes de modelos basados en la arquitectura Transformer:

- **Codificadores:** El exponente es BERT (2018). Se especializan en entender el contexto. En el mundo de los lenguajes, se usan para análisis sintáctico, detección de errores y búsqueda semántica de código.
- **Decodificadores:** La familia GPT (1, 2, 3, 4). Su foco es la generación. Han dado lugar a herramientas como GitHub Copilot, capaces de autocompletar funciones enteras basándose en un comentario.
- **Modelos de Transformación:** Como T5 y BART. Son ideales para la traducción. En programación, se aplican para transpilación (convertir código de un lenguaje a otro, como Python a C++).

Aplicación en Lenguajes y Compiladores

La técnica de NLP ha dejado de ser solo para "idiomas" y ahora se aplica formalmente a los lenguajes de programación:

A. El Transformer como Compilador Eficiente

- Investigaciones recientes (como "Transformers are Efficient Compilers, Provably") demuestran que estas redes pueden emular tareas críticas de un compilador con gran eficiencia:
- Construcción de AST (Árbol de Sintaxis Abstracta): Los Transformers pueden aprender la estructura jerárquica de un programa sin reglas gramaticales explícitas.
- Resolución de Símbolos y Análisis de Tipos: Pueden inferir tipos de variables y su alcance (scope) analizando el contexto de uso, superando en ocasiones a los analizadores estáticos tradicionales en lenguajes dinámicos.

B. Optimización de Código

Los compiladores modernos (como LLVM o GCC) están integrando modelos de atención para decidir qué optimizaciones aplicar. En lugar de usar heurísticas fijas para el unrolling de bucles o la asignación de registros, utilizan modelos entrenados para predecir qué transformación resultará en un código más rápido.

Comparativa Técnica: Era Anterior vs. Era de la Atención

Característica	Era Pre-2018 (RNN/LSTM)	Era de la Atención (Transformers)
Procesamiento	Secuencial (uno por uno).	Paralelo (toda la secuencia a la vez).
Contexto	Se "olvida" de lo lejano (gradiente desvaneciente).	Acceso directo a cualquier token (Self-attention).
Representación	Embeddings estáticos (Word2Vec).	Embeddings contextuales (BERT/GPT).
Uso en Compiladores	Limitado a detección de patrones simples.	Capaz de generar y optimizar lógica compleja.

Estado Actual y Futuro

Hoy vivimos la era de los LLMs (Large Language Models) aplicados al código, como Codex (OpenAI) o CodeT5 (Salesforce). La tendencia actual se mueve hacia:

- **Multimodalidad:** Modelos que entienden tanto la documentación (lenguaje natural) como el código ejecutable.
- **Compiladores Híbridos:** Sistemas donde el frontend sigue siendo simbólico y formal (para garantizar corrección), pero el backend de optimización es manejado por agentes de IA basados en atención para maximizar el rendimiento del hardware.

Conclusión

La evolución del Procesamiento del Lenguaje Natural y de los Diagramas T representa uno de los cambios de paradigma más profundos en la tecnología: la transición de un enfoque simbólico y prescriptivo hacia uno probabilístico y neuronal. Mientras que en sus inicios los Diagramas T se limitaban a representar la relación entre lenguajes mediante reglas gramaticales estrictas, hoy son el reflejo de sistemas complejos que "entienden" la semántica y el contexto.

La integración de conceptos como los embeddings y la arquitectura de los Transformers ha permitido que las máquinas dejen de ver el código o el lenguaje como una simple secuencia de tokens aislados para procesarlos como espacios matemáticos continuos. Este avance no solo ha optimizado la traducción automática, sino que ha dado paso a una nueva generación de "Compiladores Híbridos" y modelos como GPT o CodeT5, donde la inteligencia artificial maximiza el rendimiento del hardware y la eficiencia del desarrollo de software. En definitiva, el Diagrama T ha dejado de ser un esquema estático para convertirse en una hoja de ruta para la convergencia definitiva entre la inteligencia artificial y la ingeniería de lenguajes.

Bibliografia

Brown, P. F., Pietra, S. A. D., Pietra, V. J. D., & Mercer, R. L. (1993). *The mathematics of statistical machine translation: Parameter estimation*. Computational Linguistics.

<https://aclanthology.org/J93-2003/>

Koehn, P. (2007). *Moses: Open source toolkit for statistical machine translation*. Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics.

<https://aclanthology.org/www.mt-archive.info/ACL-2007-Koehn.pdf>

Koehn, P. (2008). *Can we and should we use translation memory for statistical machine translation?*. Proceedings of the 12th Conference of the European Association for Machine Translation. <https://aclanthology.org/W08-0510.pdf>

Llorens Agost, J. S. (s.f.). *Traducción automática estadística basada en tuplas* [Tesis de Doctorado, Universitat Politècnica de València].

<https://personales.upv.es/~jtomas/articulos/tesis.pdf>

Ortiz, A. (s.f.). *Diagramas de tumbas (T-diagrams)*.

https://arielortiz.info/s201313/tc3040/diagramas_de_tumbas.pdf

Scribd. (s.f.). *Bootstrapping in Compiler Design*.

<https://es.scribd.com/document/261154389/Bootstrapping-in-Compiler-Design>

Wang, W. (s.f.). *Syntax-based statistical machine translation*. SciSpace.

<https://scispace.com/pdf/syntax-based-statistical-machine-translation-1kcd1cemn5.pdf>

Wikipedia. (s.f.). *IBM alignment models*. https://en.wikipedia.org/wiki/IBM_alignment_models

Wikipedia. (s.f.). *Statistical machine translation*.

https://en.wikipedia.org/wiki/Statistical_machine_translation

Wikipedia. (s.f.). *Tombstone diagram*. https://en.wikipedia.org/wiki/Tombstone_diagram

Wikipedia. (s.f.). *Traducción automática estadística*.

https://es.wikipedia.org/wiki/Traducción_automática_estadística

Williams, P., & Koehn, P. (2012). *GHKM rule extraction and incremental decoding in a syntax-based statistical machine translation system*.

<https://maxapress.com/data/article/ker/preview/pdf/S026988891200029X.pdf>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. In Advances in neural information processing systems (NIPS).

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). CodeBERT: A pre-trained model for programming and natural language.

Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A survey of machine learning for big code and naturalness. ACM Computing Surveys (CSUR).